POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

# SYLABUS PRZEDMIOTU

| | |
|---|---|
| **Nazwa przedmiotu:** | **Programowanie systemowe** |
| **Kod przedmiotu:** | LLP |
| **Kierunek / Profil:** | Informatyka / praktyczny |
| **Tryb studiów:** | stacjonarny |
| **Rok / Semestr:** | 2 / 4 |
| **Charakter:** | obowiązkowy |
| **Odpowiedzialny:** | mgr inż. Michał Hyla |
| **Wersja z dnia:** | 19.02.2026 |

## 1. Godziny zajęć i punkty ECTS

| Wykłady | Ćwiczenia | Laboratori | Z prowadzącym | Praca własna | Łącznie | ECTS |
|---|---|---|---|---|---|---|
| 30 h | — | 30 h | 60 h | 40 h | 100 h | **4** |

**Zadania do samodzielnej realizacji:** Samodzielna implementacja programów systemowych; ćwiczenia z systemowych wywołań API; przygotowanie do kolokwiów; praca z narzędziami debugowania na poziomie systemu.

## 2. Forma zajęć

| Forma zajęć | Sposób zaliczenia |
|---|---|
| Wykład | Zaliczenie z oceną |
| Laboratorium | Zaliczenie z oceną |

## 3. Cel dydaktyczny

Celem dydaktycznym przedmiotu jest zrozumienie, jak komputer wykonuje program na poziomie architektury i instrukcji procesora, oraz nabycie praktycznych umiejętności

programowania w asemblerze x86. Student nauczy się świadomie operować pamięcią, stosem, przepływem sterowania (skoki, pętle) i arytmetyką, a także projektować i organizować kod

### *Wersja w języku angielskim (English version)*

*The educational objective of the course is to understand how a computer executes a program at the level of processor architecture and instructions, and to acquire practical skills in x86 assembly programming. Students learn to work consciously with memory, the stack, control flow (jumps, loops), and arithmetic, as well as to design and organize code.*

## 4. Przedmioty wprowadzające

| Przedmiot | Wymagane zagadnienia |
|---|---|
| Podstawy Programowania | Programowanie strukturalne |

## 5. Treści programowe

| Nr zaj. | Wykład | Ćwiczenia / Laboratorium / Pracownia |
|---|---|---|
| 1. | Podstawy architektury komputera | |
| 2. | Wstęp do programowania w x86 assembly | |
| 3. | Pamięć | |
| 4. | Skoki | |
| 5. | Arytmetyka | |
| 6. | ONP + Stos | |
| 7. | Pętle | |
| 8. | Projekt / funkcje | |
| 9. | Projekt / funkcje, include, asm zaawansowany | |
| 10. | Bootsector OS, BIOS | |
| 11. | 32-bit assembly | |
| 12. | Bootsector, ładowanie jądra | |
| 13. | Proste jądro | |
| 14. | Proste mechanizmy OS, rozwój jądra OS | |

## 6. Przedmiotowe efekty uczenia się

### Wiedza

- Student zna i rozumie zaawansowane pojęcia z zakresu architektury i organizacji systemów komputerowych: model von Neumanna, pipeline, pamięć podręczna (cache), architekturę wielordzeniową i zestawy instrukcji (x86-64, ARM). [K_W06]
- Student zna i rozumie zasady programowania na poziomie asemblera: reprezentacje liczb (U2, IEEE 754), arytmetykę binarną, tryby adresowania, wywołania systemowe i konwencje wywoływania funkcji. [K_W06]

### Umiejętności

- Student potrafi zaprojektować proste układy kombinacyjne i sekwencyjne, obliczyć reprezentacje liczb całkowitych i zmiennoprzecinkowych oraz napisać programy w asemblerze x86-64 realizujące zadane algorytmy. [K_U12]
- Student potrafi analizować kod maszynowy i asemblerowy wygenerowany przez kompilator C/C++, identyfikować optymalizacje i rozumieć niskopoziomowe aspekty wykonania programu. [K_U12]

### Kompetencje społeczne

- Student jest gotów do samodzielnego zgłębiania wiedzy z zakresu architektury procesorów i programowania niskopoziomowego w oparciu o dokumentację producentów i literaturę branżową. [K_K03]
- Student jest gotów do stosowania wiedzy o niskopoziomowej architekturze systemów przy projektowaniu wydajnego i bezpiecznego oprogramowania. [K_K05]

## 7. Kryteria oceny

**Ćwiczenia / Laboratorium:**

- kolokwium
- ocena pracy podczas ćwiczenia

## 8. Metody dydaktyczne

**Wykład:**

- wykład z prezentacją multimedialną
- wykład z prezentacją oprogramowania

**Ćwiczenia / Laboratorium:**

- rozwiązywanie zadań
- warsztaty

## 9. Literatura

**Podstawowa:**

- Brak danych.

**Uzupełniająca:**

- Brak danych.